

Kubernetes Cluster Deployment with EKS

An Enterprise-Grade Container Orchestration Platform on Amazon Web Services

Project Report

Submitted by

P Samuel Vijay

Roll No.: 2451-23-749-051

Department of Computer Science and Engineering

(IoT, Cybersecurity & Blockchain Technology)

4th Year

Maturi Venkata Subba Rao Engineering College (MVSREC)

Internship Domain: Cloud Infrastructure and DevOps Engineering

Abstract

This report documents the design, implementation, and evaluation of a production-style Kubernetes cluster deployed on Amazon Elastic Kubernetes Service (EKS). The project automates the provisioning of cloud infrastructure using Terraform, an Infrastructure-as-Code (IaC) tool, to create a Virtual Private Cloud, managed worker node groups, and the EKS control plane across multiple Availability Zones. A custom containerized application is built with Docker, pushed to a private Amazon Elastic Container Registry (ECR) repository, and deployed onto the cluster using native Kubernetes manifests. Traffic is exposed to the internet through an AWS Network Load Balancer, and the deployment's ability to scale is demonstrated by manually increasing pod replica count. A complementary static portfolio website was also deployed using Amazon S3 and CloudFront, with continuous deployment automated through a GitHub Actions pipeline, illustrating a broader DevOps workflow alongside the container orchestration work. All infrastructure was provisioned, tested, verified through command-line and AWS Console evidence, and subsequently torn down to avoid unnecessary cloud costs. This report presents the system architecture, technologies used, implementation methodology, testing outcomes, and results obtained, along with a discussion of the advantages, limitations, and future scope of the deployed system.

Table of Contents

1. Introduction	4
1.1 Background	4
1.2 Need for the Project	4
1.3 Cloud Service Models	4
2. Existing System	4
2.1 Limitations of the Existing Approach	4
3. Proposed System	5
4. System Architecture	5
4.1 Architecture Components	6
5. Technologies Used	6
6. Implementation Methodology	7
6.1 Infrastructure Provisioning with Terraform	7
6.2 Container Image Build and ECR Push	7
6.3 Kubernetes Deployment	7
6.4 Complementary CI/CD Pipeline (Portfolio Website)	7
7. System Testing	8
8. Results and Discussion	8
9. Advantages and Limitations	10
9.1 Advantages	10
9.2 Limitations	11
10. Applications	11
11. Future Scope	11
12. Conclusion	12
13. References	12

1. Introduction

1.1 Background

Modern software is increasingly delivered as collections of independently deployable microservices packaged as containers. While containerization solves the problem of consistent packaging, it introduces a new challenge: coordinating potentially hundreds of containers across many machines, handling failures, distributing traffic, and scaling capacity as demand changes. Kubernetes has emerged as the de facto standard for solving this orchestration problem, but operating a self-managed Kubernetes control plane is itself a significant operational burden, requiring teams to patch, secure, and scale the control plane in addition to their applications.

1.2 Need for the Project

Amazon Elastic Kubernetes Service (EKS) removes this burden by offering Kubernetes as a fully managed service: AWS operates and scales the control plane across multiple Availability Zones, while the user retains full control over worker nodes and workloads. This project was undertaken to gain hands-on, practical experience with the complete lifecycle of a cloud-native deployment on EKS — provisioning infrastructure through code, containerizing an application, securing access through identity-based permissions, and exposing the application to the public internet through a load balancer.

1.3 Cloud Service Models

Cloud offerings are typically classified into three broad service models:

- **Infrastructure as a Service (IaaS)** — provides raw compute, storage, and networking resources that the user configures (e.g. Amazon EC2, VPC).
- **Platform as a Service (PaaS)** — abstracts away the underlying infrastructure and lets users deploy applications directly (e.g. AWS Elastic Beanstalk).
- **Software as a Service (SaaS)** — delivers a complete, ready-to-use application to the end user (e.g. Gmail).

Amazon EKS sits between IaaS and PaaS: AWS manages the Kubernetes control plane as a service, while the user still provisions and manages the underlying networking and worker node infrastructure, giving this project a strong IaaS component through its use of Terraform.

2. Existing System

Traditional approaches to hosting containerized workloads rely on either manually managed virtual machines or a self-hosted Kubernetes control plane running on infrastructure the team owns and patches directly.

2.1 Limitations of the Existing Approach

- The Kubernetes control plane (API server, etcd, scheduler) must be manually secured, patched, and kept highly available.

- No built-in mechanism to automatically distribute the control plane across multiple Availability Zones.
- Scaling worker capacity in response to demand requires custom automation.
- Access control between team members and the cluster is not natively tied to existing cloud identity systems.
- Higher operational overhead diverts engineering time away from application development.

3. Proposed System

The proposed system uses Amazon EKS to offload control plane management to AWS, while automating the provisioning of all supporting infrastructure through Terraform. Key characteristics of the proposed system include:

- A managed, multi-Availability-Zone EKS control plane operated by AWS.
- Infrastructure-as-Code provisioning of the VPC, subnets, and worker node groups using Terraform.
- A private container registry (Amazon ECR) to store and serve application images.
- IAM-based authentication mapped to Kubernetes role-based access control (RBAC).
- A Network Load Balancer to distribute incoming traffic across running pods.
- The ability to scale application replicas on demand, laying the foundation for automated horizontal scaling.

4. System Architecture

The architecture is composed of a control layer, a data/compute layer, and a supporting set of AWS services that together provide identity management, image storage, and traffic routing. Figure 1 illustrates the overall system architecture and the relationships between its components.

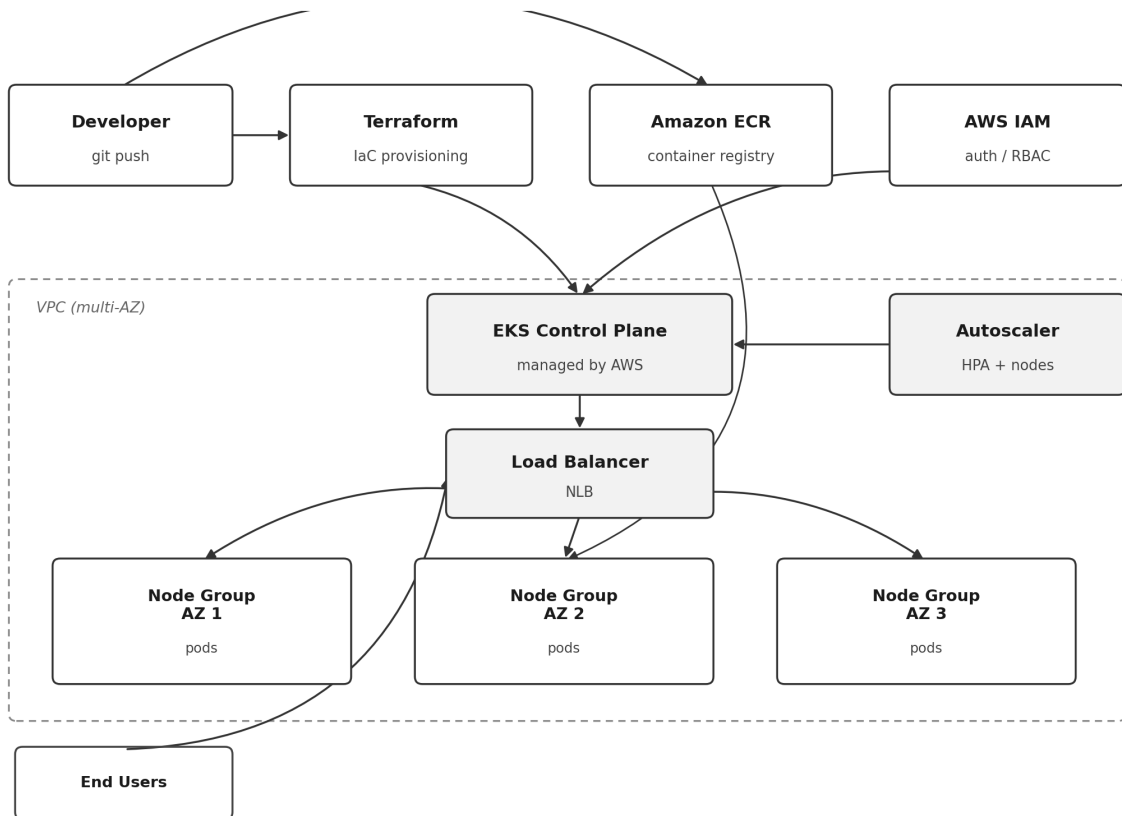


Figure 1: System architecture of the EKS cluster deployment.

4.1 Architecture Components

- **Terraform** — provisions the VPC, subnets, and managed node groups declaratively.
- **EKS Control Plane** — the managed Kubernetes API server, scheduler, and etcd store, operated by AWS across multiple Availability Zones.
- **Managed Node Groups** — EC2 instances that register with the control plane and run application pods.
- **Amazon ECR** — a private registry that stores the custom-built container image consumed by the cluster.
- **AWS IAM** — maps IAM identities to Kubernetes RBAC permissions for secure cluster access.
- **Load Balancer** — a Network Load Balancer that routes external traffic to healthy pods.
- **Autoscaler** — represents the horizontal pod and node scaling capability demonstrated through manual replica scaling in this deployment.

5. Technologies Used

Technology	Purpose
Amazon EKS	Managed Kubernetes control plane
Terraform	Infrastructure-as-Code provisioning

Amazon ECR	Private container image registry
AWS IAM	Identity and access management
Docker	Building the containerized application image
Kubernetes (kubectl)	Deploying and managing workloads on the cluster
AWS Network Load Balancer	Distributing incoming traffic across pods
Amazon S3 + CloudFront	Static website hosting and global content delivery
GitHub Actions	CI/CD automation for the portfolio website

6. Implementation Methodology

6.1 Infrastructure Provisioning with Terraform

The VPC, subnets, EKS control plane, and managed node group were defined declaratively across a set of Terraform configuration files, using the official *terraform-aws-modules* registry modules for the VPC and EKS resources. A single NAT Gateway was used, rather than one per Availability Zone, to reduce cost while preserving outbound connectivity for private subnets. The infrastructure was provisioned using the standard Terraform workflow:

- `terraform init` — downloads provider plugins and registry modules.
- `terraform plan` — previews the resources to be created.
- `terraform apply` — provisions the VPC, EKS cluster, and node group (approximately 15–18 minutes).

6.2 Container Image Build and ECR Push

A custom application image was built on top of an nginx base image, with a bundled HTML page identifying the deployment. The image was tagged and pushed to a private Amazon ECR repository created by Terraform, using the standard Docker authentication and push workflow (`docker build`, `docker tag`, and `docker push`, following authentication via `aws ecr get-login-password`).

6.3 Kubernetes Deployment

Once the cluster was reachable via `kubectl` (configured using `aws eks update-kubeconfig`), the application was deployed using two Kubernetes manifests: a **Deployment** resource specifying two replicas of the custom ECR image with defined CPU and memory resource limits, and a **Service** resource of type `LoadBalancer` exposing port 80 to the internet. Both manifests were applied using `kubectl apply -f`.

6.4 Complementary CI/CD Pipeline (Portfolio Website)

Alongside the EKS deployment, a static portfolio website was deployed to Amazon S3 with static website hosting enabled, distributed globally through an Amazon CloudFront distribution, and secured with HTTPS by default via CloudFront's managed certificate. A GitHub Actions workflow was configured to automatically synchronize website files to the S3 bucket and invalidate the CloudFront cache on every push to the main branch, demonstrating a complete, automated CI/CD pipeline independent of the Kubernetes workload.

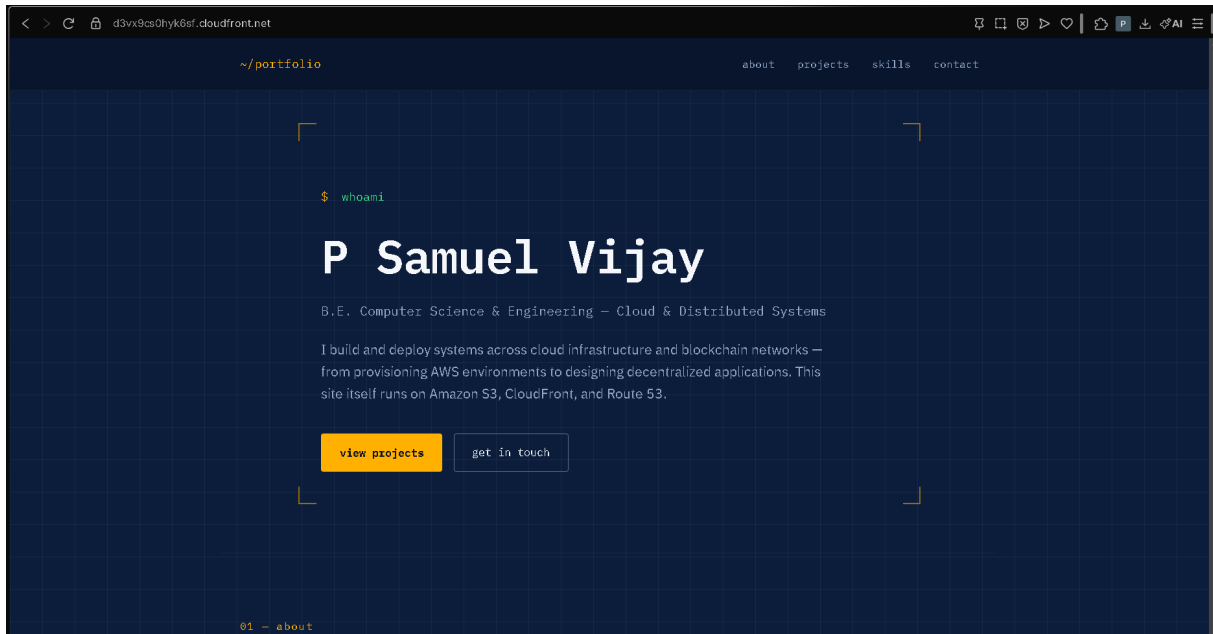


Figure 2: Portfolio website served via Amazon CloudFront, deployed through the automated CI/CD pipeline.

7. System Testing

The deployment was verified using the following test cases:

Test Case	Procedure	Expected / Observed Result
Cluster and node health	Run <code>kubectl cluster-info</code> and <code>kubectl get nodes</code> .	Control plane reachable; both worker nodes reported Ready.
Pod deployment	Run <code>kubectl get pods -o wide</code> after applying the Deployment manifest.	All pod replicas reach Running status, distributed across nodes.
Manual scaling	Run <code>kubectl scale deployment demo-app --replicas=4</code> .	Pod count increases from 2 to 4 without downtime to existing pods.
Load balancer accessibility	Run <code>kubectl get service</code> and open the reported EXTERNAL-IP in a browser.	Custom application page loads successfully over HTTP.
Container image source	Inspect the deployed pod image reference.	Image confirmed to be pulled from the private Amazon ECR repository.

8. Results and Discussion

All test cases described in Section 7 passed successfully. The following figures present direct evidence collected during the deployment, including AWS Console views and command-line output.

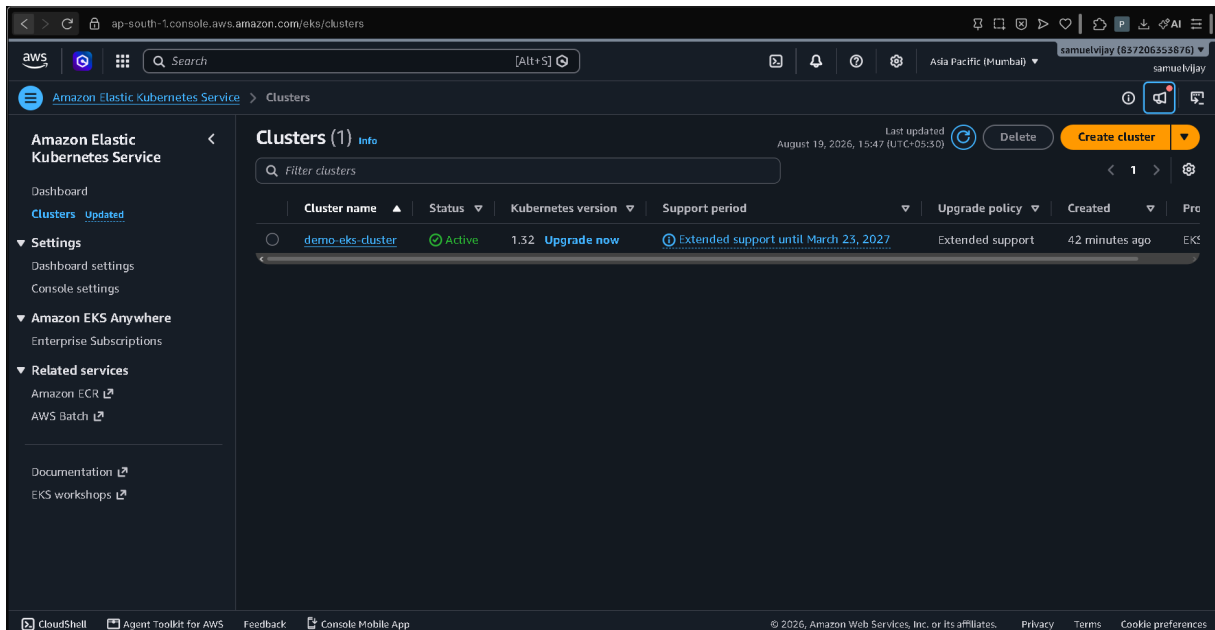


Figure 3: Amazon EKS Console showing the cluster in an Active state, running Kubernetes version 1.32.

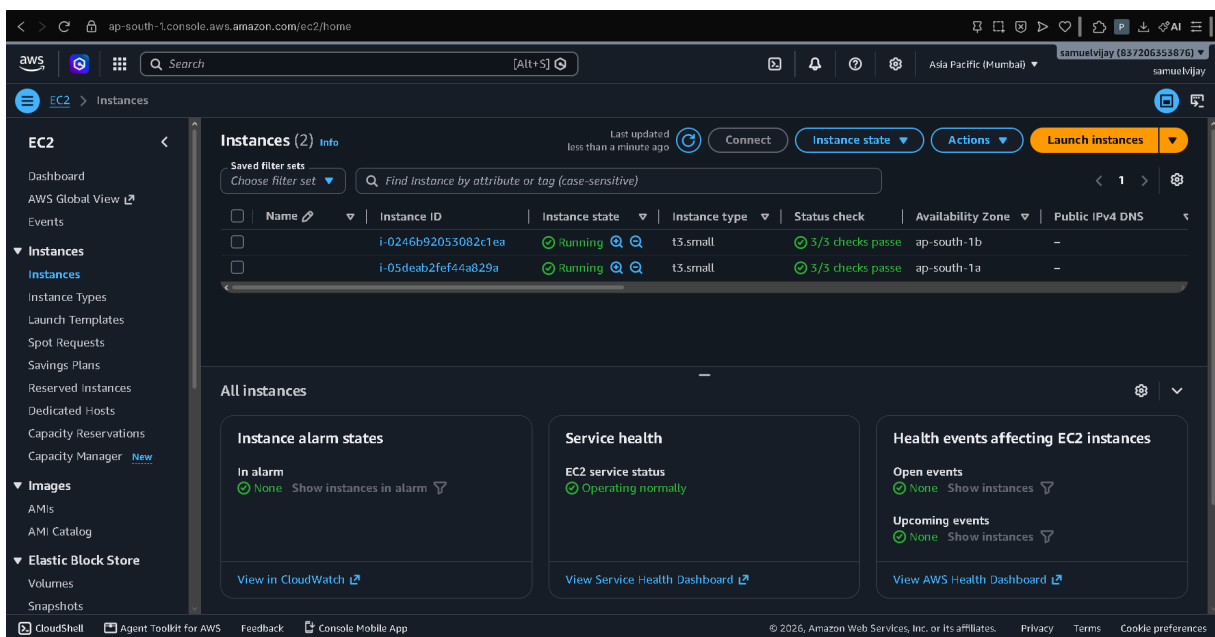


Figure 4: Amazon EC2 Console showing both worker node instances in a Running state with passing status checks.

```
PS E:\My_Projects\Blackbucks Project> kubectl cluster-info
Kubernetes control plane is running at https://28DA913DE372AE598A2848072EFF4C6.gr7.ap-south-1.eks.amazonaws.com
CoreDNS is running at https://28DA913DE372AE598A2848072EFF4C6.gr7.ap-south-1.eks.amazonaws.com/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS E:\My_Projects\Blackbucks Project> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-0-1-75.ap-south-1.compute.inte Ready    <none>   18m   v1.32.13-eks-254016e
ip-10-0-2-162.ap-south-1.compute.int Ready    <none>   18m   v1.32.13-eks-254016e
PS E:\My_Projects\Blackbucks Project> kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP              NODE                                HOSTNATED NODE   READINESS GATES
demo-app-7c6c6d8779-65lqp          1/1     Running   0           14m   10.0.1.171      ip-10-0-1-75.ap-south-1.compute.int <none>           <none>
demo-app-7c6c6d8779-9sqm           1/1     Running   0           16m   10.0.2.246      ip-10-0-2-162.ap-south-1.compute.int <none>           <none>
demo-app-7c6c6d8779-fdv25          1/1     Running   0           14m   10.0.2.178      ip-10-0-2-162.ap-south-1.compute.int <none>           <none>
demo-app-7c6c6d8779-k6ktg          1/1     Running   0           16m   10.0.1.82       ip-10-0-1-75.ap-south-1.compute.int <none>           <none>
PS E:\My_Projects\Blackbucks Project> kubectl get service demo-app-service
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
demo-app-service    LoadBalancer 172.20.138.240   afd384372f2594259a0da7a8c0289120-d00ebcb60049a3d5.elb.ap-south-1.amazonaws.com 80:30351/TCP    16m
PS E:\My_Projects\Blackbucks Project> start http://afd384372f2594259a0da7a8c0289120-d00ebcb60049a3d5.elb.ap-south-1.amazonaws.com
```

Figure 5: Command-line output confirming cluster health, four running pods (scaled from two), and a Load Balancer service with an assigned external DNS name.

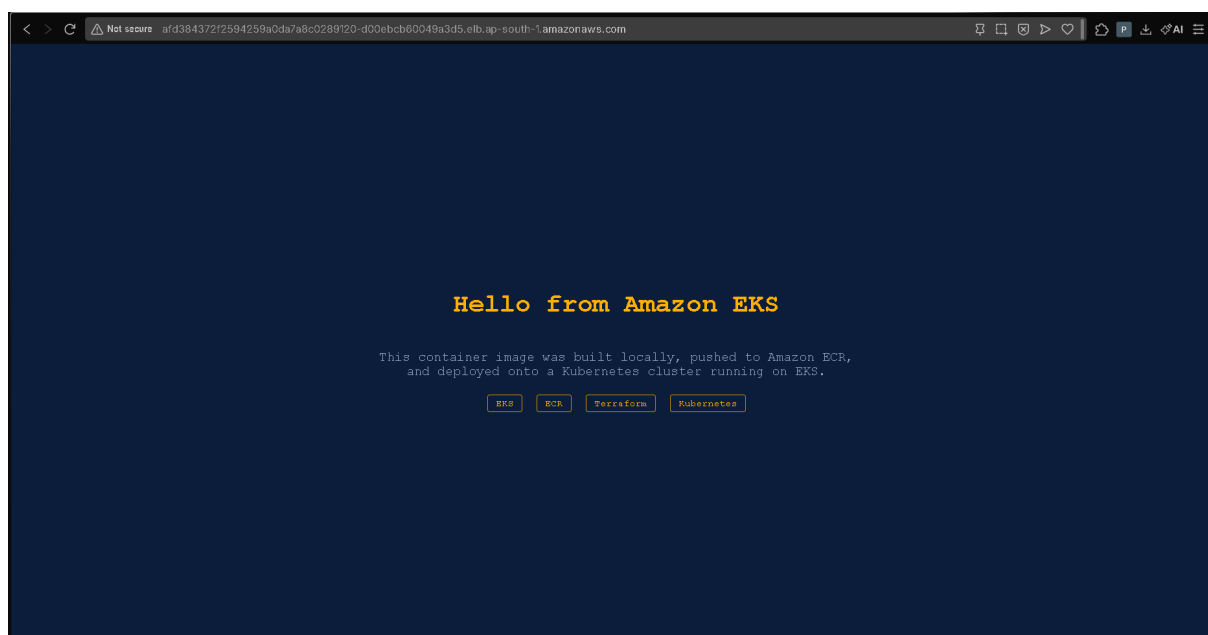


Figure 6: Custom application page, served from the container image pulled from Amazon ECR, accessed through the Load Balancer URL.

The command-line evidence in Figure 5 confirms that both worker nodes reached a Ready state and that manually scaling the deployment from two to four replicas succeeded without interrupting existing pods, demonstrating the elasticity that an automated Horizontal Pod Autoscaler would exploit under real traffic conditions. The browser output in Figure 6 confirms that traffic reaches the application correctly through the Load Balancer and that the deployed image originates from the private ECR repository, rather than a public registry.

9. Advantages and Limitations

9.1 Advantages

- Reduced operational overhead, since AWS manages, patches, and scales the Kubernetes control plane.

- High availability by default, with control plane and worker nodes spread across multiple Availability Zones.
- Elastic scalability that can match infrastructure capacity to real traffic demand.
- Strong security posture through native IAM integration and a private container registry.
- Repeatable, version-controlled infrastructure through Terraform, enabling consistent environments.

9.2 Limitations

- Amazon EKS incurs a fixed hourly control-plane charge with no free tier, in addition to worker node costs.
- A steeper learning curve is required, involving familiarity with Kubernetes, Terraform, and AWS IAM.
- The deployed Load Balancer service used plain HTTP; enabling HTTPS via a custom domain would require Route 53 and AWS Certificate Manager, which were intentionally scoped out of this deployment to avoid recurring domain and DNS costs.
- The Cluster Autoscaler and Horizontal Pod Autoscaler were represented conceptually and through manual scaling, rather than configured to react automatically to live metrics.

10. Applications

The architecture demonstrated in this project is broadly applicable to:

- Hosting microservice-based backend applications that require independent scaling of components.
- E-commerce and enterprise applications requiring high availability across multiple zones.
- Continuous deployment pipelines where new container versions must be rolled out with minimal downtime.
- Educational and demonstration environments for learning cloud-native infrastructure practices.

11. Future Scope

- Register a custom domain and configure Route 53 with AWS Certificate Manager to enable HTTPS on the application Load Balancer.
- Configure the Kubernetes Cluster Autoscaler and Horizontal Pod Autoscaler to react automatically to CPU/memory metrics rather than relying on manual scaling.
- Introduce a GitOps workflow (e.g. ArgoCD) to automate Kubernetes deployments in addition to the existing infrastructure automation.
- Add centralized monitoring and logging using CloudWatch Container Insights or a Prometheus/Grafana stack.
- Evaluate Spot Instances or Karpenter for further cost optimization of worker node capacity.

12. Conclusion

This project successfully demonstrates the end-to-end deployment of a containerized application on a managed Kubernetes cluster using Amazon EKS. By combining Terraform for infrastructure automation, Amazon ECR for secure image storage, and native Kubernetes primitives for deployment and scaling, the project delivers a working, verifiable cloud-native platform. All resources were provisioned, validated through command-line and Console-based evidence, and torn down promptly after testing to avoid unnecessary cost, reflecting a cost-conscious and disciplined approach to cloud resource management. The complementary CI/CD pipeline for the portfolio website further demonstrates broader DevOps competency beyond container orchestration alone. Overall, the project provides practical, verifiable experience with the core tools and workflows used in professional cloud infrastructure and DevOps engineering roles.

13. References

- 1 Amazon EKS Documentation — <https://docs.aws.amazon.com/eks>
- 2 Terraform AWS Provider Documentation — <https://registry.terraform.io/providers/hashicorp/aws>
- 3 terraform-aws-modules (VPC & EKS) — <https://github.com/terraform-aws-modules>
- 4 Amazon ECR User Guide — <https://docs.aws.amazon.com/AmazonECR>
- 5 Kubernetes Documentation — <https://kubernetes.io/docs>
- 6 Amazon S3 & CloudFront Documentation — <https://docs.aws.amazon.com/AmazonS3> ,
<https://docs.aws.amazon.com/AmazonCloudFront>

Project Repository: The complete source code, Terraform configuration, and Kubernetes manifests for this project are available at:
<https://github.com/psamuelvijay/aws-eks-enterprise-platform>